# NVDA

*来源文件：NVDA.docx*

电面:

1. 写 scoreboard，2 个 agent 接进来，一个 A 一个 B，分别会产生事务，顺序不确定，要求，发现一对 A 和 B 是想要的，这时候 call function check pair. 如果连续收到 2 个 A/2 个 B 则报错，或者收到一个，另一个超过 timeout 还没到，也报错

```
function void write_A(my_trans tr);
  if(trans_A != null) begin
    `uvm_error(get_type_name(), "dup trans A")
    return;
  end
  if(trans_B != null) begin
    check_pair(tr, trans_B);
    return;
  end
  trans_A = tr;
  fork
    begin
      p_a    = process::get();
      timer  = $stime();
      count_time();
    end
  join_none
endfunction
```

```
class scoreboard extends uvm_scoreboard;
  `uvm_component_utils(scoreboard)
  // ...
  my_trans trans_A;
  my_trans trans_B;
  process  p_a;
  process  p_b;
  int timeout_time = xxx;
  int timer = 0;

  task count_time;
    while(1) begin
      ##1;
      if($stime() - timer > timeout_time) begin
        `uvm_error(get_type_name(), "timeout")
        break;
      end
    end
  endtask
```

```
  function void write_B(my_trans tr);
    if(trans_B != null) begin
      `uvm_error(get_type_name(), "dup trans B")
      return;
    end
    if(trans_A != null) begin
      check_pair(trans_A, tr);
      return;
    end
    trans_B = tr;
    fork
      begin
        p_b    = process::get();
        timer  = $stime();
        count_time();
      end
    join_none
  endfunction

  function void check_pair(my_trans A, my_trans B);
    `uvm_info(get_type_name(), "xxxxxxxxx")
    trans_A = null;
    trans_B = null;
    if(p_a != null) p_a.kill(); p_a = null;
    if(p_b != null) p_b.kill(); p_b = null;
    timer = 0;
  endfunction

endclass
```

2. setup/hold violation

clk ____|^^^^|_____|^^^^|____|^^^^|____|^^^^|___

D ^^^^^^|_____

Clk 周期 200ps，D 在 150ps 的时候拉低，第一个上升沿在 100ps
Tsetup = 150ps
Thold = 100ps

问是否有任何 violation。

答案：发生了 hold violation 因为后者 FF 在 rising edge 开始接受信号，过了仅仅 50ps 信号 D 就拉低了.

follow up: 写 checker/SVA 来检查 hold 和 setup violation:

```
//setup violation
int time_0 = 0;
always @(posedge clk) begin
        if (($stime() – time_0) < T_setup) begin
                `uvm_error(get_type_name(), "setup violation")
        end
end
always @(D) begin
        time_0 = $stime();
end

//hold violation
int time_0 = 0;
always @(posedge clk) begin
        time_0 = $stime();
end
always @(D) begin
        if($stime() – time_0 < Thold) begin
                `uvm_error(get_type_name(), “hold violation”)
        end
end
```

3. constraint

```
class base;
```

```
        rand int a;
        constraint c {(a >= 0 && a<=99 }|| (a >= 201 && a <= 299);}
endclass

class child extends base;
        constraint c_0 {
        a >= 100 && a <=199;
}
Endclass
```

问题随机化 child class 会怎样？ 会失败，因为两个约束都生效，但是条件互斥。

问假如把 child class 里面的约束 c_0 注释掉，如何对 a 完全随机化，也就是让 base class 里面的约束不生效？ 用 c.constraint_mode(0);

# Clock

*来源文件：NVDA_CLK.docx*

电面:

1. 写 scoreboard，2 个 agent 接进来，一个 A 一个 B，分别会产生事务，顺序不确定，要求，发现一对 A 和 B 是想要的，这时候 call function check pair. 如果连续收到 2 个 A/2 个 B 则报错，或者收到一个，另一个超过 timeout 还没到，也报错

```
function void write_A(my_trans tr);
  if(trans_A != null) begin
    `uvm_error(get_type_name(), "dup trans A")
    return;
  end
  if(trans_B != null) begin
    check_pair(tr, trans_B);
    return;
  end
  trans_A = tr;
  fork
    begin
      p_a    = process::get();
      timer  = $stime();
      count_time();
    end
  join_none
endfunction
```

```
class scoreboard extends uvm_scoreboard;
  `uvm_component_utils(scoreboard)
  // ...
  my_trans trans_A;
  my_trans trans_B;
  process  p_a;
  process  p_b;
  int timeout_time = xxx;
  int timer = 0;

  task count_time;
    while(1) begin
      ##1;
      if($stime() - timer > timeout_time) begin
        `uvm_error(get_type_name(), "timeout")
        break;
      end
    end
  endtask
```

```
  function void write_B(my_trans tr);
    if(trans_B != null) begin
      `uvm_error(get_type_name(), "dup trans B")
      return;
    end
    if(trans_A != null) begin
      check_pair(trans_A, tr);
      return;
    end
    trans_B = tr;
    fork
      begin
        p_b    = process::get();
        timer  = $stime();
        count_time();
      end
    join_none
  endfunction

  function void check_pair(my_trans A, my_trans B);
    `uvm_info(get_type_name(), "xxxxxxxxx")
    trans_A = null;
    trans_B = null;
    if(p_a != null) p_a.kill(); p_a = null;
    if(p_b != null) p_b.kill(); p_b = null;
    timer = 0;
  endfunction

endclass
```

2. setup/hold violation

clk ____|^^^^|_____|^^^^|____|^^^^|____|^^^^|___

D ^^^^^^|_____

Clk 周期 200ps，D 在 150ps 的时候拉低，第一个上升沿在 100ps
Tsetup = 150ps
Thold = 100ps

问是否有任何 violation。

答案：发生了 hold violation 因为后者 FF 在 rising edge 开始接受信号，过了仅仅 50ps 信号 D 就拉低了.

follow up: 写 checker/SVA 来检查 hold 和 setup violation:

```
//setup violation
int time_0 = 0;
always @(posedge clk) begin
        if (($stime() – time_0) < T_setup) begin
                `uvm_error(get_type_name(), "setup violation")
        end
end
always @(D) begin
        time_0 = $stime();
end

//hold violation
int time_0 = 0;
always @(posedge clk) begin
        time_0 = $stime();
end
always @(D) begin
        if($stime() – time_0 < Thold) begin
                `uvm_error(get_type_name(), “hold violation”)
        end
end
```

3. constraint

```
class base;
```

```
        rand int a;
        constraint c {(a >= 0 && a<=99 }|| (a >= 201 && a <= 299);}
endclass

class child extends base;
        constraint c_0 {
        a >= 100 && a <=199;
}
Endclass
```

问题随机化 child class 会怎样？ 会失败，因为两个约束都生效，但是条件互斥。

问假如把 child class 里面的约束 c_0 注释掉，如何对 a 完全随机化，也就是让 base class 里面的约束不生效？ 用 c.constraint_mode(0);

Panel:

第一轮:

1. 过简历，了解 background 花了比较久

2. 经典问题，给个 dut，问怎么验。不过这个 input ouput 都是 clock，需要脑子转个弯整体不难，但细节记不住了。

3. 问了一些 CDC 的概念问题

第二轮:

1. 写一个 monitor 和 scoreboard，monitor 负责监测一个 clock 的 period，每 capture 一个 clock cycle 把这个时间打包发送给 scoreboard。scb 接了 2 个 monitor，一个提供这个 clock cycle，另一个是 reference clock cycle。两个值的差不应该超过 5%

```
class my_monitor extends uvm_monitor;
    ...

    int last_pos_edge = -1;
    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.clk)
            int current_time = $stime();
            if(last_pos_edge != -1) begin
                int cycle = current_time - last_pos_edge;
                ap.write(cycle);  //ap to scb
            end
            last_pos_edge = current_time;
        end
    endtask
endclass

class my_scoreboard extends uvm_scoreboard;
    ...

    int my_queue[$];
    task write_ref(int clk_peroid);
        my_queue.push_back(clk_peroid);
    endtask

    task write_input(int clk_peroid);
        if(my_queue.size() == 0) begin
            `uvm_error("xxxxxxxxx")
        end

        int expected_clk_peroid = my_queue.pop_front();
        if(abs(expected_clk_peroid - clk_peroid) * 100 / clk_peroid >= 5 ) begin
            `uvm_error("xxxxxxxxxx")
        end
    endtask
endclass
```

2. 写个约束，记不住具体题目了，非常非常简单那种

3. 写个 assertion，非常简单具体记不住了, 大概答案就是这样:
$fell(a) |-> !b[*2:$];

第三轮:

1. 问了一些 x-prop 相关知识

2. 假设 RTL verification 做的很完备了，是否一定还要做 gate level simulation

a) 需要，我给的答案是 RTL -> gate 这个过程是 EDA 工具自动化完成的。你不能就认为这个过程 100%正确的，你需要跑 simulation 来确认。而且这个翻译出来的 gate，本身也是可以优化的。And，EDA 工具的公司比如 synopsys 的工具现在也开始用 ai 了, ai 会犯错所以 gate level simulation 不可以省.

3. sequence 里面发起了一个 rgm 的写，问发生了什么

a) 会产生对应的 bus transaction，然后 dut 会看到并且更新 register file

b) 对于 rgm 本身，write 被 call 的一瞬间 desired value 被更新，auto predict 会更新 mirror value

c) 如果你用 uvm_do 来写 reg，你需要自己有个 monitor 去监测 bus 上发生了什么，然后创建对应的事物然后用自己写的 predictor 去更新 mirror value

4. 给了个 uvm 的 test。问为什么说找不到 test。因为没用工厂注册

a) 解释工厂注册的原理，用 uvm 源码解释的

b) 问如果 new 函数给的名字非常奇怪，没有意义的名字会不会 work

i. 会 work，但是你后面打 log 时候别人看不懂

ii. 对于 test component 也许还可以，但如果你对于某个需要被多个 component share 的 component，比如 config。你做 uvm config db 的时候会非常痛苦，因为你这个名字和 handler name/type name 不一致

5. 简单的算法题，function，输入是一个区间。多次 call 这个函数，合并区间并且去重输出全部 unique number

a) 外部创建一个 set，每次输入区间就把所有数字放到 set 里面即可

第四轮：

1. 一个 clock 会经过三个模块，每个模块会有 1 个参数并对 clock 频率做修改，你知道最终的 output clock f 和 clock input f，让写一段程序，输出一个合法的 3 个参数的组合，对于前面 2 个模块，每个模块的输出的时钟频率有一个范围。

clk_in(100Mhz) -> 模块 a, 此时除以 a_div, 输出的 clk_a 为 10-1000mhz -> 模块 b, 此时乘以 b_div, 输出为 clk_b 为 900mhz - 1.6ghz -> 经过模块 c 除以 c_div, 输出为 1000mhz.

这个一开始以为是 dut，仔细看是个数学问题，程序用随机值依次随机 a_div, b_div,由于最后的输出值是固定的，所以 c_div 是通过 a_div 和 b_div 求出的一个确定值.

a_div = $random(1, 10);

b_div = $random(9 * a_div, 16 * a_div);

c_div = ……

输入 100mhz，输出 1000mhz，说明最终输出是 10 倍的 input，用分子式推导即可

2. 写 scoreboard，check pair 是电面原题:

我说了是店面原题，但是面试官还是要面这个

# Combo

*来源文件：NVDA_combo.docx*

1. 给一段代码：

```
A = input
B = A
C = B
```

Input 值为 0 1 2 3 4 5 这样，问 ABC 三个值的变化情况(每个 cycle)

2. 改成 NBA，问有什么变化么

3. 隐藏 bug: ABC 的初始值是没给的，所以初始值是 X

2. 一段 perl 代码问你做的是什么 (不会写 perl，所以用 python 来说明题目)

```
Initial array = [1, 2 ,3 ,4 ,5, 6, 3, 4, 5, 6]
final_array = []
for num_1 in initial_array:
        matched = 0
        for num_2 in range(len(final_array)):
                If num_1 == num_2:
                        matched = 1
                        Break
        If not matched:
                Final_array.append(num_1)
```

这段代码结束以后 final_array 是 initial_array 的去重版本。

Follow up question 问如何让这段代码更高效运行?

可以用一个 set 来追踪，不用每次都 for loop 循环 final array.

3. 模块有一个 input signal 和一个 clk, 2 个 output (error_assert, clear flag). 要求设计状态机:

初始值为 IDLE 状态，当收到第一个 input signal pulse 以后，64 个 cycle 以内如果收到第二个 pulse，error_assert 要拉高，直到收到 clear_flag 然后恢复到 idle 状态。

答案： 需要一个 counter，状态机每个 clk 更新状态。需要以下状态: IDLE, ERROR_WINDOW, ERROR_ASSERT.

初始状态 IDLE 并让 counter=0，收到第一个 input signal 进入 ERROR_WINDOW，开始每个 cycle counter++。 如果 counter >= 64 重新进入 IDLE 状态. 如果 counter < 64 的时候收到第二个 input signal pulse，进入 ERROR_ASSERT 状态并拉高 error_assert. 收到 clear_flag 就拉低 error_assert 并回到 IDLE 状态。如果其他状态看到 clear_flag 则无视

4. 给个 DUT，一个 input，但是 input 里面有 channel id，对于 output，有几个 channel 就有几个 output port。问如何验证

a) 每个 port 需要一个 agent，分别连到 scb 上，每个 channel id 创建一个 queue，然后 input 事物来了进 queue，output 事物来了去 queue 里面做匹配然后校验

b) 对于这个场景写个 driver。不难，要注意的是因为不确定是否 back to back，所以每次要 try next item，如果没有则 drive_idle()

c) Follow up，如果每个 channel 是 credit based 呢？那么在 driver 里面需要每次拿到事物，对于对应的 channel id 看 credit 是否够用，如果不够用，我们需要一个 associate array of queues，key 是 channel id，然后对于 credit 不够用的事物存到对应的 queue 里面。每个 cycle 尝试驱动事物的时候先看对应的 channel 有没有

credit 恢复，如果有的话先驱动数据结构里面的事物。 同时需要另一个线程来检测 output port 是否有事物出来了，如果有的话则需要增加对应 channel 的 credit

d) Follow up，如果 channel 是不确定的呢？每次编译时候才知道有几个 channel 怎么处理？(不需要写详细代码。口述+简单代码即可)。

这种情况下只能用 fork join 来批量创建 output port 的 agent/monitor，然后在这些 agent 和 scb 之间用一个 fifo 类似的数据结构，所有的事物写入这个 fifo/mailbox，然后在 scb 那边，用一个线程不停的去从这个 fifo 里面取事物，然后做匹配/校验。 我在回答这个问题的时候没说具体用哪个数据结构，因为 analysis fifo 是不可以有多个 ap 同时往里写的，mailbox 我也不确定是否可以支持多个 port 同时往一个 mail box 里面写，所以我当时说的是类似 fifo 的数据结构然后多个 port 往里面写。

5. C++问题

a) Give a int number, set nth bit as 0

number = number & ~(1 << n);

需要注意的是，n 不可以>=32，因为在 C/C++里面你对一个 32bit 的值位移≧32 的话，这个行为是 undefined，也就是你会得到一个不确定的值但是编译器并不会报错.

b) 记不住具体问题了，但跟上面的操作差不多，简单的 bit shift

6. Vendor machine 状态机设计

a) 不是传统的写代码，要跟面试官一起，理清楚需求，以及想好怎么设计最省状态，需要哪些状态，以及状态之前如何转移

b) 大概就是机器有 2 种商品，分别是 15 美分和 25 美分的商品。用户可以投币，用户只有 5 美分和 20 美分的硬币。找零有更多选择，5，10，15，20 都可以找零。

c) 这是个开放问题，要和面试官一起设计这个贩卖机。比如是先投币还是先选商品，以及基于你的设计，有哪些可能的状态。比如如果先选商品，那么目标价格

就确定了，到达指定金额会自动执行交易并找零，所以不会出现用户一直塞硬币塞进去很多钱的情况。

d) 大致流程：

i. 先确定目标商品

ii. 开始投币，因为只有 2 种可能的投币选项，所以可以针对两种商品分别推倒出来所以可能的状态直到金额 >= 目标价。

iii. 到达目标价之后给出商品开始进入找零，逻辑和上面类似只是余额是逐渐递减的过程，优先找最大额的硬币，直到 balance 为 0

iv. 全程都是沟通+口述，没怎么写代码，写的也都是确定有哪些状态的这种 notes

v. 对于最终给商品的状态的确定能说出来 mealy/moore machine 的区别是加分项

# GPU

*来源文件：NVDA_GPU.docx*

电面：

1. 什么是 uvm phase，怎么 work 的

2. objection 是什么，怎么 work 的

3. Constraint，给定 address 和 size，要去读/写 memory。然后同时有个 range，要求不能读到这个 range 里面去（上限 4096）

```
rand bit[31:0] address;

rand int size;

Constraint c {
    address + size <= 4096;

    address > range_upper_bound || addresss + size <= range_lower_bound;

}
```

4. Follow up question，没有这个 range 了，现在规定不停的在 0-4095 这个区间里面取，直到不能取为止

加一个 queue，每次取完在 post randomize 里面加入到这个 queue 里面，然后用一个 helper function，要求每次新产生的 address + size 不能和这个 queue 里面的区间有 overlap

5. SVA

一个 input clk，一个 input int 是 expected_clock_period. Input clk 时钟周期是不稳定的，所以需要写个 assertion 和每个 expected_clock_period 相差不过 5%.

```
Int last_rising_edge = -1;

 always @(posedge clk) begin

        last_rising_edge = $stime();

end


property my_property @(posedge clk);

        last_rsing_edge != -1 |-> 100 * ($stime - last_rising_edge) /
expected_period < 5;

endproperty
```

6. Follow up question:

    expected_clock_period is not synced with clk, how to do it?

    Answer: use another variable to store expected_clock_period at each rising edge. And compare real clock period with this variable.

# Interview-NV

*来源文件：Interview-NV.docx*

First interview:

1. **The interviewer(He) asked me if there is anything I did not on the resume but I want to share.**

   I said what I did is just on my resume. The better answer would be: I did have other projects, but I think they are not very related to the role. For example, in the network processor design course, we built a multi-threaded network processor with a hardware accelerator based on bloom filter to find the malicious packet in wired-speed.( I can also share ML hardware accelerator project below, but my brain just went blank during the interview)

2. **He asked me if I have any projects on my ML hardware accelerator courses.**

   My answer was too vague and not confident. Since I write it on my resume, I should prepare to talk about at least the overview of my project. So the better answer would be:

   We first trained and quantized a CNN model called SqueezeNet on Pytorch. Then we modeled the hardware accelerator using HLS and implemented the accelerator on Xilinx ZYNQ FPGA platform to accelerate inference. (If asking for results, then answer accuracy and kernel latency(performance).)

3. **How RoB and CFC work together to allow out-of-order execution while guarantee in-order commitment**

   My explanation lacked organization. I started with RoB, but I didn't explain that well. A clearer explanation: The ROB guarantees in-order retirement even though instructions execute out of order. Instructions may finish execution in any order, but they commit from the head of the ROB in program order. This ensures precise exceptions and allows safe recovery from branch mispredictions or faults. The ROB also tracks speculative instructions and identifies which younger instructions must be flushed after a misprediction.

   The CFC handles speculative register renaming recovery. It maintains the committed RRAT state together with checkpointed speculative rename information that collectively represent the FRAT state. During dispatch, instructions are renamed using speculative mappings to eliminate WAR and WAW hazards. When a branch is dispatched, a checkpoint is created. If the branch later mispredicts, the processor restores the checkpoint associated with that branch and frees all younger speculative checkpoints.

4. **Why do we need two kinds of RAT, what they are solving**

   The FRAT (Front-end RAT) tracks the latest speculative register mappings and is used during dispatch for register renaming. It allows the processor to eliminate WAR and WAW hazards by assigning new physical registers to destination operands.

   The RRAT (Retirement RAT) stores only committed mappings, so it represents the precise architectural state of the machine.

   We need both because speculative instructions may later be flushed due to exceptions or branch mispredictions. In that case, the processor must recover the last correct architectural state quickly. The RRAT provides this committed mapping, while the FRAT may contain speculative mappings from wrong-path instructions.

5. **Then he asked what is the purpose of CFC and what is it solving in the tomasulo machine**

   I think it's because I didn't explain well in my previous question.

6. **Describe the key difference between MOESI vs simpler MESI protocol. What problem does that O state solve and how to improve performance**

   MOESI protocol adds an additional "dirty" state called O state.

   The O state allows a cache line to be both dirty and shared at the same time.

   In MESI, when another core reads a line in Modified state, the dirty data usually has to be written back to memory (or memory updated) before the line can become shared.

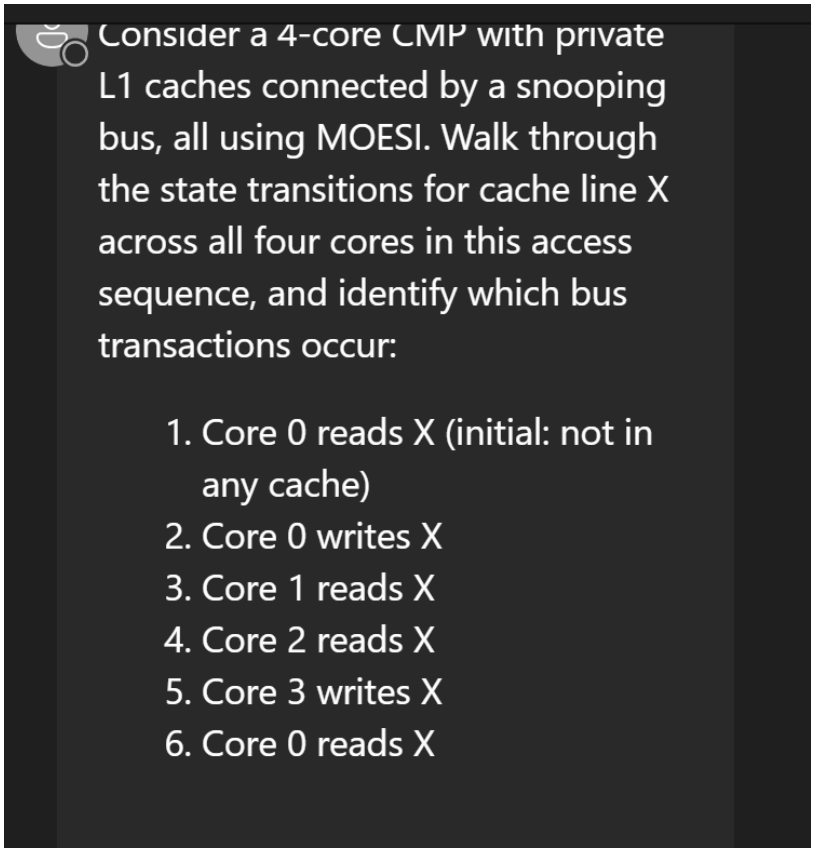
   In MOESI, the cache holding the dirty line transitions from M to O and directly supplies the data to the requesting cache through cache-to-cache transfer. The requester enters S state while memory may still contain stale data.

   This reduces memory writebacks and memory bandwidth usage, improving performance and latency.

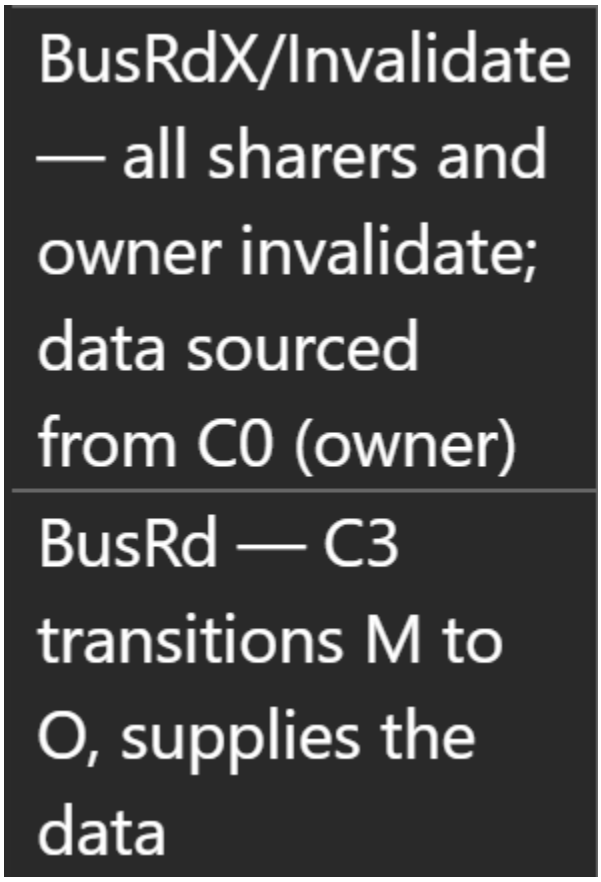
7. **What is the difference between a particular processor owing the line in exclusive state versus modified state**

   I didn't say the point. Exclusive state is the "clean" state that has the only copy of a cache line, while Modified state is the "dirty" state that has the only copy of a cache line.

8. **Fill the state transition table for each step using MOESI and MESI respectively**



| Bus Transaction |
| --- |
| BusRd (no other sharer, line clean from memory) |
| None (silent upgrade E to M) |
| BusRd — C0 supplies data via cache-to-cache, does not write back |
| BusRd — C0 (owner) supplies the data |




It’s ok for me to fill the table. But I didn’t answer the follow up question correctly: **which step should write back to main memory using MESI protocol.**

In MESI, the writeback occurs in step 3 and step 6.

In step 3, Core0 holds the cache line in Modified state. When Core1 reads the line, the line transitions from M to S. Since Shared state represents a clean shared copy, memory must become up-to-date before both caches can hold the line in S state.

Similarly, in step 6, Core3 holds the line in Modified state. When Core0 reads the line, Core3 transitions from M to S, so the modified data must again be written back (or memory updated) before the line becomes shared.

# 补充题目

*手动录入的文字题+ 截图整理*

## 一、DDR Memory Allocator UVM Transaction 题目（文字题）

写一个 UVM transaction / sequence item，用来描述一个 DDR memory allocator 的随机配置。要求如下：

- DDR 的分配粒度是 64 bytes。
- 总 allocated memory size 在 128KB 到 16MB 之间。
- 最多支持 8 个 tile，但实际存在的 tile 数量可以是 1 到 8 个。
- 只有存在的 tile 才会被分配 memory。
- 每个存在的 tile size/depth 在 128KB 到 1MB 之间。
- 所有存在的 tile 都必须完全落在 allocated memory range 内。
- 任意两个存在的 tile 之间不能 overlap。
- tile 的物理顺序可以是乱的，不能默认 tile0 一定在 tile1 前面。

要求把这些条件写进 UVM transaction 里的 random variables 和 constraints。

## 二、面试真题回忆（截图整理）

**内容主要靠回忆题目顺序比较杂乱**

### 第一轮

1. 简历相关
    a. Cpu side 能否修改 cache 内容？（不因为 read only）
    b. 怎么保证两个 transaction map 到同一个 set？（保证 (addr >> byteoffset) %num_set 相等）
    c. PLRU
2. Uvm 的启动流程
3. Uvm top module 里面包含什么？（clock，dut，interface 之类的）
4. 一个 transaction class 已经定义了默认约束。如何在运行时针对某一次随机化覆盖默认约束，而不修改原来的 class（soft keyword 和 inline constraint）
5. 怎么判断 dut 是不是验证完整

### 第二轮

1. 给了一个 dut
    a. 写一个可以在 test 里面定义是读或者写的 sequence（sequence 里面用 inline constraint，然后在 test 里面设置参数）
    b. 写一个 test 包含并行的两个 sequence 一个 write 一个 read（fork join）
        i. 怎么在 test 里面保证并行的 read 和 write sequence 遵循先 write 再 read 的顺序？（用 semaphore 进行线程的同步）
    c. 写一个 instruction 的 transaction，然后写了一些简单的 constraint 比如说规定多少到多少位是 op code，还有整体的范围有多大
        i. 在这个 transaction 里面写一个 covergroup
        ii. 如果 operand 来自 forwarding path，怎么修改 transaction？（加入 instruction id 然后 monitor 在根据同一个 id 填 transaction）

## 第三轮

1. 简历，项目的一些组件的实现细节并且现场写代码进行说明

    a. 各讲一下 ahb 和 apb 的特点和区别

    b. 有哪些 harzard? （data，control，structural）

    c. 写了一些简单的 constraint

    d. 写 SVA 检查 ahblite burst 不能跨过 1kb 的边界

```
property p_no_1kb_cross;
   int unsigned beats;

   @(posedge HCLK) disable iff (!HRESETn)

   (
      HREADY &&
      HTRANS == 2'b10, // NONSEQ, 新 burst 开始

      beats = (HBURST == 3'b011) ? 4  : // INCR4
              (HBURST == 3'b101) ? 8  : // INCR8
              (HBURST == 3'b111) ? 16 : 1
   )
   |->
   (
      int'(HADDR[9:0]) +
      beats * (1 << HSIZE) - 1 <= 1023
   );
endproperty
```

2. 写 SVA 检查时钟 clk 的频率是否为 100MHz

```
property check_clock_period(realtime expected_period);
   realtime previous_edge_time;

   @(posedge clk)
   (1, previous_edge_time = $realtime)
   |=>
   ($realtime - previous_edge_time == expected_period);
endproperty

assert property (check_clock_period(10ns));
```

3. 一个接口连续传输 packet stream，除了 data，还需要哪些信号才能正确区分 packet（sop，eop 和 valid）

## 第四轮

1. 纯 coding 基本上是用 sv 写 constraint 和 function

    a. 约束一个 100 个 bit 类型的动态数组的前 8 个元素的和等于 6 还有一些简单的 constraint

    b. 写一个 function 返回这个数组所有连续十个 1 的起始位置

    c. 约束一个动态数组让每次随机化产生的数字在某一个范围只出现一次

    d. Inline constraint 相关

    e. 还有一些我记不太清楚了不过都是基础题，真题都有覆盖到